

Campus Ride Management Platform

Design Document

Real-Time Campus Mobility and Ride Management Platform

Java 21 · Spring Boot · PostgreSQL · React + TypeScript + Vite · WebSocket/STOMP · Leaflet / OpenStreetMap

TEAM MEMBERS

Samyak Mahapatra 23116087

Chintaluri N V S M S Rajeev Lochan 23116028

Nandipati Lalith 23116066

Dara Zenas Zephaniah 23321010

1. Problem Understanding

IIT Roorkee's sprawling campus relies on e-rickshaws for last-mile connectivity. Ride requests and driver availability are currently coordinated through fragmented, informal channels — phone calls, word-of-mouth, and manual queuing — resulting in:

- Inefficient resource utilisation and long passenger wait times
- No real-time visibility of driver availability
- Uneven demand distribution across campus zones
- Zero feedback loop between riders and drivers
- No audit trail for rides or analytics for planning

The goal is to build a centralised, web-based ride management platform that mirrors the engineering challenges of commercial ride-hailing systems — real-time communication, geospatial coordination, state synchronisation, and multi-user workflows — while remaining lightweight and maintainable by a small team.

2. System Architecture

2.1 Architectural Overview

The system follows a three-tier architecture: presentation layer (React SPA), application layer (Spring Boot REST + WebSocket), and data layer (PostgreSQL).

Layer	Technology	Responsibility
Frontend	React 18, TypeScript, Vite, Leaflet	SPA — UI, state management, WebSocket client
Backend API	Java 21, Spring Boot 3, Spring Security	REST endpoints, business logic, JWT auth
Real-Time	WebSocket + STOMP (Spring)	Pub/sub topics for live updates
Database	PostgreSQL (JPA / Hibernate)	Persistent storage, ACID transactions
Auth	Spring Security + JWT	Stateless authentication, role-based access

2.2 Component Interaction

Client ↔ Backend uses two channels: **HTTP REST** (Vite proxy on `/api`) for request/response operations such as login, ride creation, and dashboard queries; and **WebSocket over STOMP** (`/ws`) for all push-based real-time events — ride state changes, driver location updates, and availability broadcasts. The `/ws` endpoint is permitted without authentication; REST APIs use JWT tokens. The frontend maintains a single persistent WebSocket connection per session, subscribing to STOMP topics based on the user's role.

3. Database Schema

The PostgreSQL schema is managed by Hibernate DDL-auto (update strategy).

3.1 Core Tables

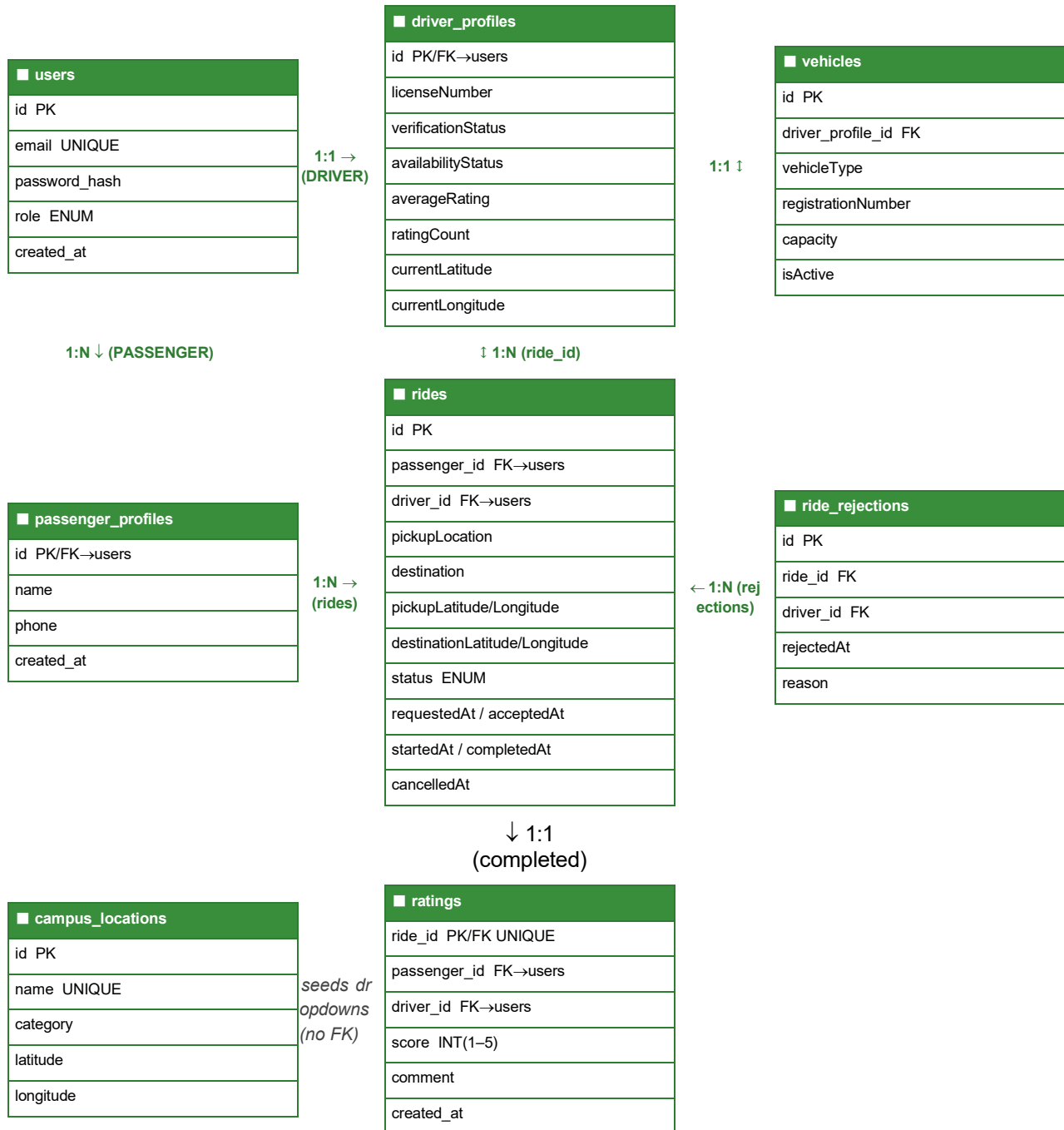
Table	Key Columns	Notes
users	id, email, password_hash, role (PASSENGER DRIVER), created_at	Base identity for all users
driver_profiles	id, user_id (FK), licenseNumber, verificationDocument, verificationStatus, availabilityStatus, averageRating, ratingCount, currentLatitude, currentLongitude	Extended driver info; coordinates updated via GPS heartbeat. Vehicle info in separate
passenger_profiles	id, user_id (FK), name, phone, created_at	Extended passenger info linked to users
vehicles	id, driver_profile_id (FK), vehicleType, registrationNumber, capacity, isActive	Vehicle details per driver; separated for normalisation
rides	id, passenger_id (FK→users), driver_id (FK→users, nullable), pickupLocation, destination, pickupLatitude, pickupLongitude, destinationLatitude, destinationLongitude, status (ENUM), requestedAt, acceptedAt, startedAt, completedAt, cancelledAt	Central table. Coordinates embedded directly — no FK to campus_locations. Status
campus_locations	id, name, category, latitude, longitude	Seeded campus POIs for dropdown population. Not FK-referenced by rides.
ride_rejections	id, ride_id (FK), driver_id (FK), rejectedAt, reason	Tracks driver rejections; prevents re-offering to same driver and supports re-broadcast
ratings	id, ride_id (FK unique), passenger_id (FK), driver_id (FK), score (1–5), comment, created_at	One rating per completed ride

3.2 Key Design Decisions

- **Pessimistic row locking:** Ride acceptance uses pessimistic locking (`findByIdForUpdate`) inside a `@Transactional` block to prevent double-assignment. If the ride is already non-REQUESTED at lock time, the operation fails with 409 Conflict.
- **Embedded ride coordinates:** Pickup/destination lat/lng are stored directly on rides, not as FKs to `campus_locations`, enabling free-form locations and avoiding JOIN overhead.
- **Soft availability:** `availabilityStatus` on `driver_profiles` minimises JOIN overhead for the common query path.
- **Audit timestamps:** `requestedAt`, `acceptedAt`, `startedAt`, `completedAt`, `cancelledAt` enable future analytics without schema changes.
- **Rejection tracking:** `ride_rejections` prevents re-offering a ride to drivers who declined it.

4. Entity Relationship Diagram

The diagram below shows the primary entities and their relationships.



PK = Primary Key

FK = Foreign Key

1 = One

N = Many

5. API Overview

5.1 REST Endpoints

Method	Endpoint	Description
POST	/api/auth/register/passenger	Register a new passenger account
POST	/api/auth/register/driver	Register a new driver account
POST	/api/auth/login	Authenticate and return JWT
GET	/api/users/me	Return authenticated user's profile
PUT	/api/users/me	Update passenger profile
POST	/api/drivers/availability/online	Set driver status to AVAILABLE
POST	/api/drivers/availability/offline	Set driver status to OFFLINE
GET	/api/drivers/available	List currently available drivers
GET	/api/drivers/requests	List pending ride requests for this driver
GET	/api/drivers/dashboard	Return ride stats, history, and ratings
POST	/api/drivers/location	Receive GPS coordinates and broadcast via WebSocket
GET	/api/locations	Return seeded campus pickup/destination POIs
POST	/api/rides	Create a new ride request
GET	/api/rides/my	Return all rides for the authenticated user
GET	/api/rides/{rideId}	Return single ride detail
POST	/api/rides/{rideId}/accept	Driver accepts ride (pessimistic-locked, atomic)
POST	/api/rides/{rideId}/reject	Driver rejects ride (recorded in ride_rejections)
POST	/api/rides/{rideId}/start	Driver marks ride as IN_PROGRESS
POST	/api/rides/{rideId}/complete	Driver marks ride as COMPLETED
POST	/api/rides/{rideId}/cancel	Passenger or driver cancels ride
POST	/api/ratings	Submit rating for a completed ride

5.2 WebSocket Topics (STOMP)

Connection endpoint: `ws://localhost:8099/ws` — no JWT required on the WebSocket handshake; auth is enforced at the REST layer.

Topic	Payload / Purpose
/topic/drivers/availability	Broadcast when any driver goes online or offline
/topic/drivers/location	Broadcast driver GPS coordinates (lat/long + driver ID)
/topic/rides/requests	Broadcast when a new ride request is created
/topic/rides/{rideId}	Unicast ride state change events (ACCEPTED, IN_PROGRESS, etc.)
/topic/rides/{rideId}/driver-location	Stream real-time driver position to passenger map
/topic/users/{userId}/notifications	Per-user push notifications (assigned, cancelled, etc.)
/topic/drivers/{driverId}/dashboard	Push dashboard stat updates after ride events

6. Design Decisions

6.1 Real-Time Communication Strategy

WebSocket with STOMP was chosen over SSE or polling because it offers full-duplex communication for driver GPS heartbeats and ride state pushes, built-in pub/sub semantics that map naturally to ride IDs and user IDs, and native Spring integration (`spring-boot-starter-websocket`) that reduces boilerplate.

6.2 Atomic Ride Assignment

Ride acceptance uses **pessimistic row locking** (`findByIdForUpdate`) inside a `@Transactional` block to prevent double-assignment. If the ride status is already non-REQUESTED at lock time, the operation fails fast with a 409 Conflict, preserving data integrity without complex distributed locking.

6.3 JWT Stateless Authentication

Stateless JWT tokens are used to authenticate all REST API calls, eliminating server-side session storage and simplifying horizontal scaling. The WebSocket endpoint (`/ws`) is currently permitted without authentication — JWT is not passed as a query parameter on the WebSocket URL in the current implementation.

6.4 Frontend State Management

React's built-in `useState` and `useContext` hooks were used instead of Redux/Zustand to keep the bundle small and data flow transparent. WebSocket events mutate component state directly via callbacks registered on topic subscriptions.

6.5 Map Integration

Leaflet with OpenStreetMap tiles was chosen over Google Maps to avoid API key management and billing. Campus POI coordinates are seeded into `campus_locations` and embedded directly into ride records at request time. Driver live positions are streamed via WebSocket; the frontend updates the map marker on movement exceeding ~12 metres or approximately every 10 seconds.

7. Feature Completeness Summary

Feature	Status
User Authentication (JWT, role-based)	■ Done
Driver Availability (online/offline/busy)	■ Done
Ride Request & Assignment Workflow	■ Done
Advanced Ride Scheduling	■ Done
Real-Time Updates (WebSocket / STOMP)	■ Done
Complete Ride Lifecycle (5 states)	■ Done
Driver Dashboard with Charts	■ Done
Ratings & Feedback System	■ Done
Live Map (Leaflet + OSM)	■ Done
Real-Time Driver GPS Tracking	■ Done
Passenger Profiles	■ Done
Vehicle Management (separate table)	■ Done
Ride Rejection Tracking	■ Done

Not Implemented (Out of Scope)

The following features were identified as optional scope and are not included in the current version:

Feature	Notes
Digital Payments	In-app fare calculation and payment gateway integration
Demand Analytics	Historical heatmaps and peak-hour visualisation
Demand Forecasting	ML-based prediction of high-demand zones and times